GIT QUEST | 30-DAY GIT LEARNING SERIES

Day 9: git restore and git reset

Week 2: Undo and Branch | Level: Absolute Beginner

Created by Rutuja Naik

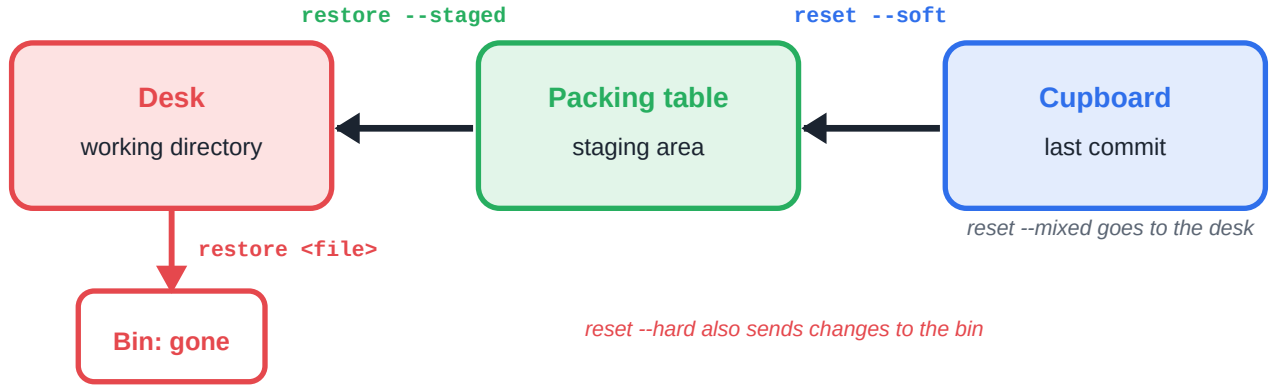
Today's mission

Everyone makes mistakes. Today you learn the most reassuring skill in Git: how to take things back, and how to pick the **right** undo for each situation.

- Unstage a file without losing your work
- Throw away edits you do not want with `git restore`
- Undo a commit with `git reset` (soft, mixed and hard)
- Know which undo is safe and which is dangerous

1. The big idea: undo moves things backwards

Every undo moves something **backwards** through the 3 areas you already know: cupboard, packing table, desk, and finally the bin.



Which undo do I need?

Situation	Use	Safe?
I edited a file and want those edits gone	**git restore <file>**	No: edits are lost
I staged the wrong file	**git restore --staged <file>**	Yes
I committed too early but want to keep my work	**git reset --soft HEAD~1**	Yes
I committed junk and want it gone completely	**git reset --hard HEAD~1**	No: work is lost

2. The 3 reset modes

`git reset HEAD~1` takes the **last commit out of the cupboard**. The mode decides where its changes go.

Mode	Command	Commit	Its changes end up...
Soft	git reset --soft HEAD~1	Undone	Staged on the packing table
Mixed (default)	git reset HEAD~1	Undone	Unstaged on the desk
Hard	git reset --hard HEAD~1	Undone	Deleted

> **Warning**
> `git restore <file>` and `git reset --hard` cannot be undone for work you never committed. Run `git status` and `git diff` first.

3. Step by step

Step 1: Save a one-line story

```
$ git init -b main undo-lab
$ cd undo-lab
$ echo "Line 1" > story.txt
$ git add story.txt
$ git commit -m "Add story"
```

Step 2: Oops! Throw away a bad edit

```
$ echo "Oops, wrong line" >> story.txt
$ git restore story.txt
$ cat story.txt
Line 1
```

Step 3: Unstage without losing work

```
$ echo "Line 2" >> story.txt
$ git add story.txt
$ git restore --staged story.txt
$ git status
Changes not staged for commit:
        modified:   story.txt   (your edit is kept)
```

Step 4: Undo a commit (soft)

```
$ git add . && git commit -m "Add line 2"
$ git reset --soft HEAD~1
$ git status
Changes to be committed:
        modified:   story.txt
```

Step 5: Undo a commit (mixed)

```
$ git commit -m "Add line 2"
$ git reset HEAD~1
Unstaged changes after reset:
M       story.txt
```

Step 6: Undo a commit (hard, careful!)

```
$ git add . && git commit -m "Add line 2"
$ git reset --hard HEAD~1
HEAD is now at 7a3c9e2 Add story
```

Bonus: bring back an older version of one file

```
$ git restore --source=HEAD~1 story.txt
```

The file returns as it was one commit ago, and appears as a normal edit you can review and commit.

Safety rules

1. **Look first:** run git status and git diff so you know exactly what you will lose.
2. restore <file> and reset --hard cannot be undone for unsaved work.
3. Only reset commits that are still on your computer. Once commits are shared (Week 3), rewriting them causes trouble for others.
4. Reset commits can usually still be rescued with git reflog (Day 26).

4. Cheat sheet

Command	What it does
git restore <file>	Discard edits (danger)
git restore --staged <file>	Unstage, keep edits
git restore --source=HEAD~1 <file>	Get an older version of a file
git reset --soft HEAD~1	Undo commit, keep changes staged
git reset HEAD~1	Undo commit, keep changes on the desk
git reset --hard HEAD~1	Undo commit, delete changes (danger)

Your Day 9 commit
File: `days-1-to-9-summary.md`
Message: `docs(day09): add days 1-9 summary and undo notes`

5. Mermaid diagram GitHub draws for you

```
```mermaid
flowchart LR
  C["Cupboard (commit)"] -- "reset --soft" --> T["Packing table"]
  C -- "reset --mixed" --> D["Desk"]
  C -- "reset --hard" --> X["Deleted"]
```
```

6. Challenge and quiz

Challenge: make two commits. Undo the last one with `--soft`, then commit again with a better message.

Q1. Which command throws away unstaged edits in a file?

Q2. What is the difference between reset --soft and reset --hard?

Q3. Which is safe: restore --staged or plain restore?

Answers: 1) git restore <file>. 2) --soft keeps the changes staged, --hard deletes them. 3) restore --staged is safe: your edits stay.

> **Coming up: Day 10**
> **git commit --amend.** Fix your last commit in one command.